GSS Religiosity and Media Pipeline Report

Data: General Social Survey, NORC at the University of Chicago (citation in the README). Every figure can be reproduced with one Docker command (README) and the queries in dbt/analyses/.

1. Extracts and variable mapping

Both cuts were built in the GSS Data Explorer with the same 22 variables and exported as JSON to source/data/raw/, unchanged (steps in the README).

Extract	Years	Records	Why
gss_cut_a_2016_2018	2016, 2018	5,215	the waves with the most answers for the religiosity and media questions, and every variable in scope available
gss_cut_b_2018_2022	2018, 2022	5,892	keeps 2018 for comparison and adds 2022 to compare with recent years

The cuts share the whole 2018 wave: 2,348 respondents (45.0% of A, 39.9% of B), with identical answers, giving 8,759 distinct respondents (limit 10,000). Each has a year the other lacks. Cut B is above "roughly 5,000" because whole waves were kept: sampling inside a wave would break the weights and ballot design. A respondent is a respondent-year (year, id_); id_ restarts each wave, so the waves are separate samples.

"Answered" is the share of people with a usable response each year. The GSS splits people into three groups (ballots a, b and c), and some questions have no answers from a whole group.

Question	Why this variable?	Answered 2016 / 2018 / 2022	Limitations and how handled
Working status	wrkstat records whether the person was working, retired, unemployed, studying or in another status last week	99.9 / 99.9 / 99.8%	Almost everyone has a usable response
Worked for at least one year	evwork asks about having worked as long as one year; wrkstat gives the employment context	36 / 35 / 37%	Almost no current workers have an answer (2 of 5,205), so this cannot be answered for them. Their records stay marked not applicable and are not counted as "no"
Education	educ records the highest year of school completed	99.7 / 99.9 / 99.4%	Measures years of schooling, not degrees. Q3 combines them into five bands
Industry	indus10 identifies the industry of the current or last job	96 / 96 / 94%	Q3 uses wrkstat to keep current workers
Job satisfaction	satjob records satisfaction with work	62 / 74 / 70%	Also answered outside the working group. Q4 keeps workers only
Life after death	postlife asks directly whether the person believes in life after death	90 / 90 / **45%**	In 2022, 1,707 responses are inapplicable and the cause is unconfirmed. They are left out of the YES/NO percentage and reported separately
Prayer frequency	pray directly measures how often the person prays	99 / 99 / 99%	"Never" is a usable response and stays in the distribution
Religion raised in	relig16 records the religion the person was raised in	99 / 99 / 98%	"None" is a usable response
Newspaper reading	news measures how often the person reads newspapers	66 / 66 / 67%	No usable answers on ballot c. Percentages use usable responses, with their count shown
TV viewing	tvhours measures daily hours watching TV	66 / 66 / 66%	No usable answers on ballot c. Zero hours is valid; missing values are left out of averages

Question	Why this variable?	Answered 2016 / 2018 / 2022	Limitations and how handled
Technology interest	inttech measures interest in technologies	48 / 50 / **0%**	Answers in 2016 and 2018; unavailable in 2022. Measures interest, not actual use

Also extracted: relig, attend, reliten, wwwhr, year, id_, ballot, the weight wtssnrps, age (89 means "89 or older"), sex and region.

Records where a question did not apply or was not asked are never dropped or recoded. Each answer is a row with a response_type: answer, inapplicable (.i), unavailable_that_year (.y), dont_know, no_answer, skipped_on_web, or not_asked_on_ballot. All come straight from the GSS code except not_asked_on_ballot, which is **derived**: .i on a ballot where nobody answered that year, unverified until checked against the questionnaires. Shares use real answers only, with the not-asked counts alongside.

2. Data profile

Written before the pipeline by profile_extracts.py. docs/data_profile_summary.md has record counts and the share of answers, special codes and blanks for every variable and year, for each cut and for the combined distinct respondents. The main points:

Year	Respondents	postlife missing	reliten missing	news missing	wwwhr missing	inttech missing
2016	2,867	9.7%	1.0%	34.3%	44.6%	51.7%
2018	2,348	9.6%	1.5%	33.6%	42.0%	50.0%
2022	3,544	55.5%	50.4%	33.3%	36.8%	100.0%

All other variables are at least 93% answered in every year, except evwork, satjob and tvhours (see Section 1 table). These are unweighted shares of special codes, not the share of eligible people who failed to answer.

- **Special codes vs answers:** no value is blank; every missing value is a label starting with a dot and a letter (.i: Inapplicable, .d: Do not Know/Cannot Choose, .n, .s, .y). Real answers never start that way, and literal answers such as "None" or "Never" are answers, not missing.
- **Asked only in some years or ballots:** news, tvhours, wwwhr have no answers on ballot c; inttech is answered on ballot a and half of b in 2016–18 and is .y in 2022; postlife and reliten are answered by about half of each 2022 ballot. These are observed patterns, not proof of what each ballot asked.
- **Effect on what can be answered:** Q4 and Q6 rest on two thirds of each wave; technology interest cannot be compared with 2022; 2022 belief in life after death rests on 1,576 answers, not about 3,200; lifetime work cannot be reported for working respondents.

3. Database and schema

DuckDB through dbt-duckdb: a single local file with no server, account or cost; a Python package, so it needs nothing beyond the pinned environment; columnar, so the counts, shares and cross-tabulations the analyses need are fast; and supported by dbt. Each layer has its own schema, so it is clear where a table comes from and the dbt layers can be rebuilt from raw.

Layer / Stored As	Purpose And Justification
raw / tables	The source exactly as received, one row per respondent per extract; never changed in place
staging / view	Typed keys and weight, one row per respondent-year, answer labels unchanged. Small and cheap to recompute
intermediate / view	One row per answer, classified against the seeds; independent of the six questions
marts / tables	dim_respondent, fct_response and fct_variable_coverage, read by every analysis; a full rebuild is proportionate at this size
reference / seeds	Version-controlled code meanings, response types and observed coverage

4. Model and structures

- **Layers:** raw → staging → intermediate → marts, with the reference seeds alongside (see Section 3).

- **Grain and keys:** dim_respondent has one row per respondent-year (8,759), key respondent_year_id (e.g. 2018-1042), with ballot, weight, demographics and work status. fct_response has one row per respondent-year per variable (157,662 = 8,759 × 18). Tests check both grains and that counts hold through every layer.
- **Encodings:** answers are readable labels (Several times a day), with the code and a number where useful. ref_codes, from the export's code list, gives each code's label and response type; ref_variable_validity, derived from the answers, records which year and ballot each question was answered in. Models join to these seeds rather than repeat them.
- **Missing values:** the source value is kept, response_type says what kind it is, and is_answer is true only for real answers (see Section 1).
- **Naming** (docs/naming_conventions.md): lowercase snake_case; stg_, int_, dim_, fct_, ref_, assert_ and q<n>_ prefixes; one schema per layer.
- **Adding a variable:** add it to the extracts and the answer_variables list and rebuild the seeds; no model changes.
- **Weights (bonus):** wtssnrps (post-stratified, adjusted for non-response, in all three waves). Shares and means are weighted; group sizes are unweighted counts.

5. Libraries and frameworks

Python 3.12 standard library for reading and validation (ast.literal_eval, because the export is Python-literal rather than strict JSON; never eval), hashlib and unittest; DuckDB; dbt-core with dbt-duckdb. **Docker** packages the environment: the base image is fixed by digest (Debian, Python 3.12.10) and every package is pinned, so a run next month matches today's. A no-cache build reproduced the run, tests and results exactly. Rejected: pandas (DuckDB does the bulk loading), pytest (unittest needs no extra package), Airflow/Dagster/Prefect (three steps in a row need no scheduler; a Python runner does it), Terraform and AWS services (the core pipeline needs no cloud; see the production sketch). Orchestration, configuration and logs are in the README.

6. Results and caveats

Each respondent is counted once: overlap_audit.sql reconciles 5,215 + 5,892 − 2,348 = 8,759, and both marts hold 8,759. Shares and means are weighted; counts are not. No standard errors or significance tests were calculated, and associations are not causes.

Q1. Prayer by religion raised in. Raised Protestant (4,408 answers): 34% pray several times a day, 14% never. Catholic (2,798): 29% once a day, 16% never. No religion (878): 43% never. Muslim (67): 68% several times a day. *Trust:* good for the three large groups; groups under about 100 are indicative only.

Q2. Belief in life after death.

Year	answered	believe	inapplicable (.i)	don't know / no answer
2016	2,588	79.5%	0	279
2018	2,123	80.1%	0	225
2022	1,576	82.2%	1,707	261

None are .y or not_asked_on_ballot. *Trust:* 2016 and 2018 are solid. In 2022 the 1,707 .i rows are spread evenly across ballots, consistent with a split form the extract does not show, so they are probably "not asked", but .i alone cannot prove it. A 2-point rise on 1,576 answers is not strong evidence of change.

Q3. Most common industry by education (working respondents). Under 12, 12 and 13–15 years of school: construction (15%, 11%, 7%). 16 and 17+ years: elementary and secondary schools (8%, 16%). *Trust:* the leading industry holds only 7–16% of each group; counts are unweighted; years of school are not degrees.

Q4. TV and job satisfaction (working respondents). Respondents, mean TV hours and share watching 4+ hours: very satisfied 1,522, 2.45, 19.3%; moderately satisfied 1,284, 2.47, 18.3%; a little dissatisfied 300, 2.08, 14.9%; very dissatisfied 98, 1.85, 11.0%. *Trust:* no difference between the two large groups; less TV among the dissatisfied rests on small groups and is not adjusted for age, hours worked or year.

Q5. Ever worked 12+ months, among people not working. 88.6% (2,880 yes, 280 no): retired 98.8%, keeping house 82.3%, other 85.2%, in school 61.5%. *Trust:* good, but 379 of 386 unemployed people are .i, so this describes the other four groups.

Q6. Newspaper and TV, wave to wave.

Wave	reads paper daily	never reads	mean TV hours	4+ hours
2016	20.9%	38.0%	2.78	25.7%
2018	17.9% (−3.0)	44.6% (+6.6)	2.73 (−0.05)	25.6% (−0.1)

Wave	reads paper daily	never reads	mean TV hours	4+ hours
2022	14.0% (−3.9)	54.7% (+10.1)	3.07 (+0.34)	29.1% (+3.5)

Trust: the newspaper decline is large and consistent. The 2022 TV rise is less certain: four years apart and partly collected on the web, so a mode effect is possible. Each wave rests on ballots a and b (1,555–2,365 answers). Online news is not measured.

7. Efficiency

Materialisation. Staging and intermediate are views: read once per build, nothing to store. Marts are tables, because every analysis reads them. At the full file (about 75,700 respondents × 7,000 variables) the long model and fct_response would become incremental tables, built one survey year at a time.

Query plan. Q1 is one of the most expensive queries (about 30 ms): two scans of fct_response, a join to dim_respondent, a group-by and a window total (Appendix A). Each scan filters on variable_name, and DuckDB skips a block of about 122,880 rows only when that value falls outside the block's min/max.

Change: fct_response is now sorted by variable_name. At 157,662 rows there are two blocks: both scans read both before and one now, too small to change the time.

At scale, I would demonstrate it this way: copy fct_response 100 times (about 15.8 million rows), once in load order and once sorted by variable_name. Run Q1 with EXPLAIN ANALYZE on both, compare the blocks read and the run time, and check that the results are identical. I would expect far fewer blocks to be read after sorting, because each variable sits in only a few blocks. A copy tests only the layout: repeated rows behave differently from 7,000 varied variables.

A new extract. The loader skips unchanged files by fingerprint and loads only the new one, in its own transaction. The seeds are rebuilt with build_reference_seeds.py; a dbt test fails if this is forgotten. dbt then rebuilds every model, cheaper at this size than incremental logic. A failed dbt build can leave the marts partly rebuilt; rerunning repairs them.

Not built, as disproportionate here: incremental models, partitioned storage, a scheduler, CI and a realistic full-scale benchmark.

8. Limitations and mitigations

Limitation	Mitigation	still open
.i means both "not on ballot" and "did not apply"	derived not_asked_on_ballot where a whole ballot has no answers	unverified against the questionnaires
2022 postlife, reliten half missing within every ballot	kept as inapplicable, reported in Q2	the cause (likely a form split) is not in the extract
inttech absent in 2022	marked .y; not compared across waves	no recent answer
"If working, worked 12+ months": almost no answers from workers	reported as unanswerable; nothing guessed	needs a job-tenure question
Education is years, not degree	five bands in Q3	add degree
Separate samples; 2022 mixed mode	descriptive comparisons only (see Q6)	–

9. Risk and complexity at scale

At the full file, values grow about 3,400× (157,662 → about 530 million).

1. **Breaks first: reading the export.** The reader parses each file in memory (0.8 s for 3 MB). At about 26 bytes per value, the full file would be roughly 13 GB. Fix: CSV or Parquet, read in a stream, one wave per file.
2. **Next: the generated SQL.** The long model writes one union all per variable; 7,000 is too many. Fix: DuckDB UNPIVOT.
3. **Grows fastest: the long table**, to about 530 million rows, too much to rebuild on every run. Fix: incremental models by survey year, kept sorted by variable.
4. **Seeds outgrow seeds:** ref_codes from 637 rows to about 200,000. Fix: load code metadata as a raw table.

5. **More sources:** each brings its own codes, missing values and weights, and comparison needs a mapping per shared concept per source (5 sources × 50 concepts = 250). The single-file database allows one writer at a time. Fix: one staging model per source into the same long shape, and a tested concept-mapping table (docs/production_sketch.md).

10. Assumptions and divergence

Assumptions: (year, id_) identifies a respondent-year; shared respondent-years are identical (checked: 0 conflicts); only .y means "not in that year"; waves are separate samples. Divergences from the design note (reasons in its section 8): no wide view for the six questions; a validity seed derived from the data; the derived not_asked_on_ballot type; demographics and work status in dim_respondent; weighted shares; a coverage mart; fct_response sorted by variable; Docker packaging.

11. AI use and human oversight

AI tools and contribution. I used ChatGPT and Claude extensively throughout the project, including design, implementation, testing, troubleshooting, analysis and documentation. AI contributed substantially to the end-to-end solution.

My contribution and oversight. I explored the GSS Data Explorer, chose the survey years, and set up and ran the pipeline on Windows and in Docker. I inspected execution results, record counts and overlap, reviewed proposed changes and the report, and questioned variable choices, missing-value handling, performance and unnecessary complexity.

12. Time spent and further work

About 16 hours, twice the 8 suggested: much of it went on understanding the requirements, learning the GSS Data Explorer, becoming familiar with the tools, and understanding and reviewing the code and checking results. With more time, I would first verify the remaining survey rules and weighting assumptions, and independently check selected results. For larger-scale use, I would test performance and incremental processing, automate tests through CI, and add failure alerts. I would also implement and evaluate the proposed AWS design.

Appendix A. Q1 Query Plan Comparison

Query: prayer frequency by religion raised in (q1_prayer_by_religion_raised.sql).

Purpose: check whether sorting the response table helps DuckDB read less data.

What the query does

The steps run in this order (DuckDB 2.0, EXPLAIN ANALYZE):

Step	Purpose
Table scans	Read prayer and religion-raised responses from fct_response, and respondents from dim_respondent
Hash joins	Match records using respondent_year_id
Group by	Count responses by religion raised and prayer frequency
Window calculation	Calculate percentages within each religion and answer-status group
Sort	Arrange the final output

The change and the result

fct_response was sorted by variable_name, survey_year, so answers to the same question are stored together. DuckDB can then skip blocks (its "row groups") that do not hold the requested variable.

On the real data (157,662 rows), each fct_response scan read both blocks before the change and one of two after it. The query takes only about 30 ms, so the data is too small to show a reliable time difference. Section 7 explains how I would demonstrate it at scale.

The larger test used a 100× copy of the data (15.8 million rows):

Measurement	Before sorting	After sorting
Blocks read per `fct_response` scan	2 of 2	1 of 2

Evidence

The full before-and-after plans are in docs/q1_query_plan.txt.